

OBSERVACIONES DE LA PRÁCTICA

Juan José Peñaloza Cod 202421523
Sarah González Almario Cod 202521379
Andres Jose Velasco Mejia Cod 202610866

Ambientes de pruebas

	Máquina 1	Máquina 2	Máquina 3
Procesadores	MacBook Air – Apple M4 (10 núcleos)	11th Gen Intel Core i7-1165G7 (4 nucleos, 8 hilos)	AMD Ryzen 7 7445HS w/ Radeon 740M Graphics (3.20 GHz)
Memoria RAM (GB)	16 GB	15.7 GB	16 GB
Sistema Operativo	macOS 26.6.2	Windows 11 Home Single Language 10.0.26200	Windows 11 Home Single Language 25H2

Tabla 1. Especificaciones de las máquinas para ejecutar las pruebas de rendimiento.

Máquina 1

Resultados para Queue con Array List

Porcentaje de la muestra	enqueue (Array List)	dequeue (Array List)	peek (Array List)
0.50%	0.015	0.014	0.004
5.00%	0.065	0.104	0.000
10.00%	0.143	0.218	0.000
20.00%	0.274	0.527	0.000
30.00%	0.429	0.845	0.000
50.00%	0.690	1.842	0.000
80.00%	1.166	3.828	0.000
100.00%	1.446	5.848	0.000

Resultados para Stack con Array List

Porcentaje de la muestra	push (Array List)	pop (Array List)	top (Array List)
0.50%	0.011	0.019	0.001
5.00%	0.119	0.151	0.000
10.00%	0.304	0.273	0.000
20.00%	0.891	0.679	0.000
30.00%	1.980	1.119	0.001

50.00%	4.547	2.421	0.001
80.00%	10.671	4.737	0.001
100.00%	15.583	6.667	0.001

Resultados para Queue con Linked List

Porcentaje de la muestra	enqueue (Linked List)	dequeue (Linked List)	peek (Linked List)
0.50%	0.016	0.018	0.001
5.00%	0.130	0.100	0.000
10.00%	0.216	0.216	0.000
20.00%	0.418	0.440	0.000
30.00%	0.669	0.658	0.000
50.00%	1.141	1.059	0.000
80.00%	1.743	1.700	0.001
100.00%	12.232	2.221	0.001

Resultados para Stack con Linked List

Porcentaje de la muestra	push (Linked List)	pop (Linked List)	top (Linked List)
0.50%	0.011	0.019	0.000
5.00%	0.096	0.149	0.000
10.00%	0.193	0.297	0.000
20.00%	0.387	0.600	0.000
30.00%	0.620	0.943	0.000
50.00%	1.244	1.574	0.001
80.00%	1.776	2.411	0.000
100.00%	2.164	3.019	0.000

Máquina 2

Resultados para Queue con Array List

Porcentaje de la muestra	enqueue (Array List)	dequeue (Array List)	peek (Array List)
0.50%	0.066	0.026	0.005
5.00%	0.142	0.167	0.001
10.00%	0.662	0.913	0.001
20.00%	0.895	1.638	0.001
30.00%	1.210	1.958	0.001

50.00%	3.437	5.939	0.002
80.00%	5.098	13.623	0.004
100.00%	7.550	20.816	0.003

Resultados para Stack con Array List

Porcentaje de la muestra	push (Array List)	pop (Array List)	top(Array List)
0.50%	0.018	0.021	0.001
5.00%	0.172	0.215	0.001
10.00%	1.038	1.278	0.001
20.00%	2.066	1.524	0.001
30.00%	2.970	2.962	0.001
50.00%	9.764	6.541	0.003
80.00%	28.878	17.051	0.003
100.00%	33.326	12.805	0.002

Resultados para Queue con Linked List

Porcentaje de la muestra	enqueue (Linked List)	dequeue (Linked List)	peek Linked List)
0.50%	0.026	0.022	0.001
5.00%	0.253	0.270	0.001
10.00%	0.478	0.530	0.001
20.00%	1.026	1.619	0.001
30.00%	2.501	2.185	0.006
50.00%	3.856	3.814	0.003
80.00%	4.263	4.310	0.002
100.00%	5.289	10.444	0.002

Resultados para Stack con Linked List

Porcentaje de la muestra	push (Linked List)	pop (Linked List)	top(Linked List)
0.50%	0.019	0.034	0.001
5.00%	0.224	0.349	0.001
10.00%	0.435	0.668	0.001
20.00%	3.722	3.109	0.007
30.00%	2.783	4.402	0.002
50.00%	3.639	4.937	0.001

80.00%	5.079	6.075	0.004
100.00%	99.635	16.192	0.005

Máquina 3

Resultados para Queue con Array List

Porcentaje de la muestra	enqueue (Array List)	dequeue (Array List)	peek (Array List)
0.50%	0.044	0.040	0.003
5.00%	0.229	0.287	0.003
10.00%	0.321	0.325	0.001
20.00%	0.555	0.725	0.003
30.00%	1.253	1.579	0.002
50.00%	1.253	2.332	0.001
80.00%	2.376	5.801	0.001
100.00%	2.810	6.174	0.002

Resultados para Stack con Array List

Porcentaje de la muestra	push (Array List)	pop (Array List)	top(Array List)
0.50%	0.021	0.024	0.002
5.00%	0.316	0.288	0.002
10.00%	0.351	0.470	0.001
20.00%	0.684	0.907	0.002
30.00%	1.076	1.201	0.001
50.00%	1.552	1.877	0.001
80.00%	3.141	2.966	0.001
100.00%	2.896	3.701	0.001

Resultados para Queue con Linked List

Porcentaje de la muestra	enqueue (Linked List)	dequeue (Linked List)	peek Linked List)
0.50%	0.040	0.024	0.004
5.00%	0.189	0.204	0.002
10.00%	0.278	0.330	0.002
20.00%	0.529	0.718	0.002
30.00%	1.272	1.184	0.001
50.00%	1.719	1.993	0.001

80.00%	2.995	4.053	0.002
100.00%	2.915	5.591	0.001

Resultados para Stack con Linked List

Porcentaje de la muestra	push (Linked List)	pop (Linked List)	top(Linked List)
0.50%	0.038	0.043	0.003
5.00%	0.333	0.407	0.002
10.00%	0.430	0.473	0.001
20.00%	0.815	0.983	0.002
30.00%	1.090	1.326	0.002
50.00%	1.628	2.075	0.001
80.00%	5.682	3.168	0.003
100.00%	3.290	3.830	0.001

Preguntas de análisis

1. ¿Se observan diferencias significativas entre las implementaciones con ArrayList y LinkedList para las funciones de Queue y Stack? ¿Cuál es más eficiente en cada operación? ¿Por qué una implementación es más rápida en ciertos casos?

Sí, hay diferencias claras y dependen de la operación. Las pruebas se ejecutaron sobre Data/GoodReads/books.csv con 10.000 registros y 44 pruebas unitarias aprobadas. En Queue, Array List agrega al final con costo $O(1)$ amortizado y fue el más estable en enqueue (de 0.015 ms al 0.50% a 1.446 ms al 100%), pero dequeue debe desplazar los elementos restantes ($O(n)$) y creció de 0.014 ms a 5.848 ms. Linked List elimina el primer nodo sin desplazamientos ($O(1)$) y su dequeue solo llegó a 2.221 ms al 100%, es decir 2.6 veces más rápido que Array List. En Stack, Linked List fue claramente mejor porque push y pop operan sobre el primer nodo: 2.164 ms y 3.019 ms al 100%, frente a 15.583 ms y 6.667 ms de Array List, que debe desplazar elementos al insertar o eliminar en la primera posición. peek y top son $O(1)$ en ambas implementaciones y sus tiempos se mantuvieron entre 0.000 y 0.004 ms. El valor de 12.232 ms en enqueue con Linked List al 100% es una anomalía experimental observada en esta corrida (la tendencia previa era 1.743 ms al 80%) y se conserva tal como se midió, sin interpretarlo como comportamiento general.

2. ¿Cuándo es preferible usar ArrayList o LinkedList? Si insertamos y eliminamos con frecuencia, ¿qué estructura conviene más? Si accedemos aleatoriamente a elementos, ¿cuál es más eficiente?

Array List conviene cuando se requiere acceso aleatorio por índice, recorridos secuenciales y operaciones al final de la estructura, ya que el acceso por posición es $O(1)$ y la contigüidad en memoria mejora la localidad de caché. Linked List conviene cuando predominan inserciones y eliminaciones en el inicio (o en un extremo conocido), porque esas operaciones son $O(1)$ y no

requieren desplazar los elementos restantes. Si insertamos y eliminamos con frecuencia al inicio, como en dequeue, push y pop de esta práctica, conviene Linked List: los tiempos al 100% lo confirman (2.221 ms en dequeue frente a 5.848 ms, y 2.164 ms en push frente a 15.583 ms). Si el acceso es aleatorio por índice, conviene Array List, porque Linked List debe recorrer los nodos uno por uno hasta la posición buscada ($O(n)$) y además consume memoria adicional para los apuntadores. Ninguna implementación es mejor en todos los casos: la elección depende de la operación dominante en el caso de uso.

3. Durante la ejecución de las pruebas ¿Se presentan anomalías en los tiempos de ejecución que no se explican con la teoría?

Sí. Con muestras pequeñas (0.50% y 5.00%) los tiempos son tan bajos que quedan por debajo de la resolución práctica del reloj y se reportan como 0.000 ms al redondear a tres decimales, especialmente en peek y top; esto no significa costo nulo, sino una medición por debajo de la precisión disponible. También influyen el calentamiento del intérprete, la carga del sistema operativo, los procesos en segundo plano, la administración de memoria y el redimensionamiento interno de las estructuras, lo que produce variaciones entre corridas. La anomalía más notoria es el enqueue de Linked List al 100%, con 12.232 ms frente a 1.743 ms al 80%: rompe la tendencia y no se explica por la teoría, por lo que se reporta como anomalía experimental y no se elimina ni se sustituye. En conclusión, los tiempos medidos corresponden a esta máquina y a esta ejecución, y complementan, pero no reemplazan, el análisis de complejidad asintótica.

4. Complete la siguiente tabla de acuerdo con qué operación es más eficiente en cada implementación (marque con una x la que es más eficiente). Adicionalmente, explique si este comportamiento es acorde con lo enunciado en la teoría. Justifique las respuestas.

		Array List	Linked List	Justificación
QUEUE	Enqueue()	x		Teoría: $O(1)$ amortizado en Array List al agregar al final; $O(n)$ en Linked List si recorre hasta el último nodo. Empírico: Array List más estable (1.446 ms al 100%); Linked List registró 12.232 ms, anomalía experimental.
	Dequeue()		x	Teoría: $O(n)$ en Array List por el desplazamiento; $O(1)$ en Linked List al eliminar el primer nodo. Empírico: 5.848 ms vs. 2.221 ms al 100%, acorde con la teoría.
	Peek()	x	x	Teoría: $O(1)$ en ambas. Empírico: 0.000–0.004 ms, dentro de la resolución del reloj.
STACK	Push()		x	Teoría: $O(n)$ en Array List al insertar en la primera posición; $O(1)$ en Linked List. Empírico: 15.583 ms vs. 2.164 ms al 100%, acorde con la teoría.
	Pop()		x	Teoría: $O(n)$ en Array List al eliminar la primera posición; $O(1)$ en Linked List. Empírico: 6.667 ms vs. 3.019 ms al 100%, acorde con la teoría.
	Top()	x	x	Teoría: $O(1)$ en ambas. Empírico: 0.000–0.001 ms en las dos implementaciones.

